

CS224N | Transformer

Selene

August 21, 2026

1 Foundations

Let $w_{1:n}$ denote a sequence of n words, where $w_i \in V$ and V is a finite vocabulary. The entire sequence can also be represented as a matrix of one-hot vectors:

$$w_{1:n} \in \mathbb{R}^{n \times |V|}.$$

A language model predicts the current word from the preceding words:

$$w_t \sim \text{softmax}(f(w_{1:t-1})),$$

where

$$f(w_{1:t-1}) \in \mathbb{R}^{|V|}.$$

The softmax converts the model output into a probability distribution over the vocabulary.

For $A \in \mathbb{R}^{\ell \times d}$, softmax is usually computed along the last dimension:

$$\text{softmax}(A)_{i,j} = \frac{\exp(A_{i,j})}{\sum_{j'=1}^d \exp(A_{i,j'})}.$$

The same rule applies to higher-dimensional tensors. For example, if $B \in \mathbb{R}^{m \times \ell \times d}$, softmax is still computed along the last dimension.

Word embeddings

Let $E \in \mathbb{R}^{d \times |V|}$ be an embedding matrix. It maps a one-hot vector w to a d -dimensional word vector:

$$x = Ew \in \mathbb{R}^d.$$

For an entire sequence,

$$x_{1:n} = w_{1:n} E^T \in \mathbb{R}^{n \times d}.$$

Contextualized representations

The word embedding Ew_i is a **non-contextualized representation**: it depends only on the word w_i , so the same word has the same representation in every context.

A contextualized representation h_i , by contrast, depends on both the word and its context. It can be written as $h_i = f(x_{1:n})$. A language model usually uses only prefix information, giving $h_i = f(x_{1:i})$.

- A **non-contextualized representation** depends only on the word itself.
- A **contextualized representation** depends on both the word and its surrounding context.

2 A Minimal Self-Attention Architecture

Broadly speaking, attention is a soft lookup mechanism over a key-value store. Given a query, it retrieves the values associated with the most similar keys. Here, “retrieve” and “most similar” mean taking a weighted average over all values, assigning greater weight to values whose keys more closely resemble the query. In self-attention, the same set of elements is used to construct the queries, keys, and values.

2.1 Key-Query-Value Self-Attention

We begin with one of the most widely used attention mechanisms: key-query-value self-attention.

For token x_i in the sequence $x_{1:n}$, three projection matrices $W_Q, W_K, W_V \in \mathbb{R}^{d \times d}$ produce its query, key, and value:

$$q_i = W_Q x_i, \quad k_j = W_K x_j, \quad v_j = W_V x_j.$$

Their roles are as follows:

- The **query** q_i represents the information that the current token is looking for.
- The **key** k_j represents the information that token x_j can offer.
- The **value** v_j represents the information that token x_j actually provides.

First, the dot product of q_i and k_j measures the relevance between x_i and x_j . A softmax then converts these scores into attention weights:

$$\alpha_{ij} = \frac{\exp(q_i^\top k_j)}{\sum_{j'=1}^n \exp(q_i^\top k_{j'})}.$$

The weight α_{ij} indicates how much token x_i should attend to token x_j . Finally, a weighted sum over all values produces the contextualized representation of x_i :

$$h_i = \sum_{j=1}^n \alpha_{ij} v_j.$$

The core self-attention process can therefore be summarized as follows:

Compute Q, K, and V → compare each Query with the Keys → apply softmax to obtain weights → take a weighted sum of the Values.

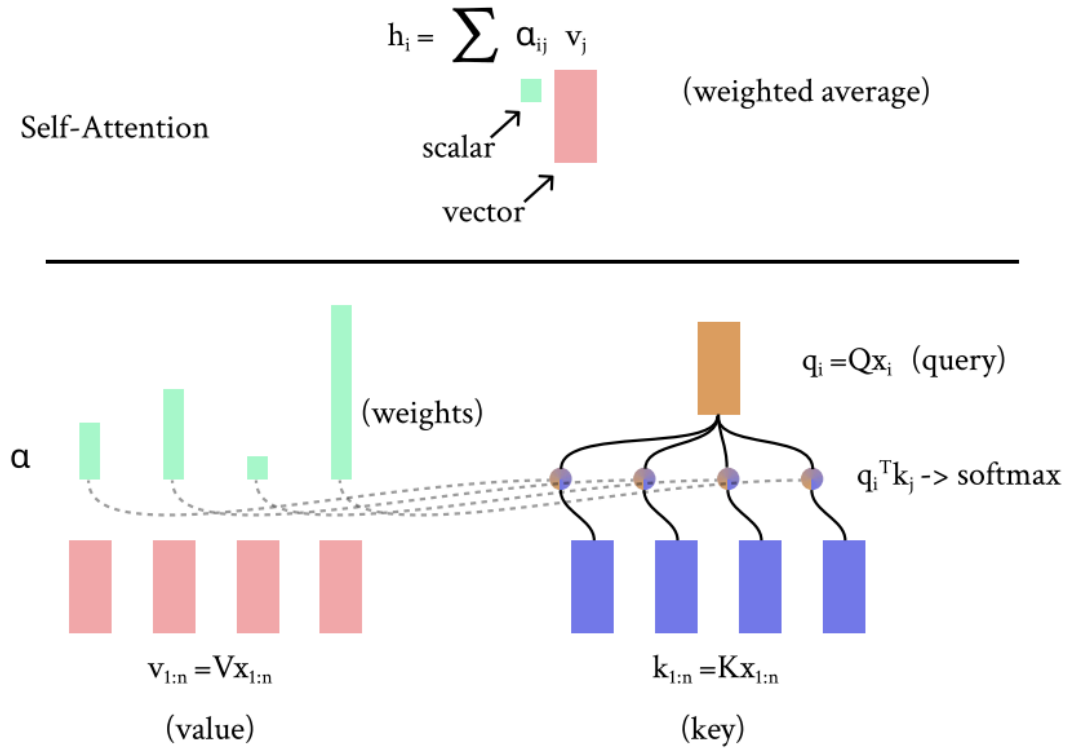


Figure 1: The key-query-value self-attention mechanism

2.2 Positional Representations

Self-attention has no built-in notion of sequence order. If we only rearrange the input tokens, the similarities and weighted sums in the attention computation are rearranged in the same way. Self-attention alone therefore cannot determine where a token occurs in the sequence.

In other words, an ordinary word embedding $x_i = Ew_i$ depends only on the token itself and not on its position.

Positional embeddings

One common approach is to learn a vector for each position. Let $P \in \mathbb{R}^{N \times d}$ be a positional embedding matrix, where N is the maximum sequence length supported by the model.

For the token at position i , add its positional vector P_i to the word embedding x_i :

$$\tilde{x}_i = x_i + P_i.$$

Self-attention then operates on $\tilde{x}_i$. The same word receives a different input representation at different positions, allowing the model to perceive sequence order.

Adding a positional bias to attention

Instead of modifying the input representation, we can modify the attention score directly so that attention itself incorporates positional information.

One approach adds a relative-position bias to the original attention scores:

$$\alpha_i = \text{softmax}(K^T q_i + b_i).$$

Here, b_i is determined by the relative positions of the tokens. Tokens closer to the current position often receive a larger bias, encouraging the model to focus more strongly on nearby information.

2.3 Element-wise Nonlinearity

If we only stack self-attention layers, every layer recomputes its attention weights. However, if we focus on the value-aggregation step while treating those weights as fixed, the result can still be expressed as a weighted linear combination

of the input representations. Attention alone does not provide all the rich nonlinear transformations expected from a deep network.

For example, the output of a second self-attention layer can be written schematically as

$$\begin{aligned} o_i &= \sum_{j=1}^n \alpha_{ij} V^{(2)} \left(\sum_{k=1}^n \alpha_{jk} V^{(1)} x_k \right) \\ &= \sum_{k=1}^n \alpha_{ik}^* V^* x_k, \end{aligned}$$

where $V^* = V^{(2)} V^{(1)}$, and α_{ik}^* represents the combined effect of the attention weights in the two layers. Its form still resembles a single self-attention layer: linearly transform the inputs and then take a weighted sum.

For this reason, self-attention is usually followed by an **element-wise feed-forward network (FFN)** that introduces additional nonlinearity:

$$h_{\text{FF}} = W_2 \text{ReLU}(W_1 h_{\text{self-attention}} + b_1) + b_2.$$

The FFN independently applies the same transformation to every token in the sequence. Different positions share parameters, but their computations do not interact within the FFN.

An FFN typically expands the hidden dimension d to a larger intermediate dimension and then projects it back to d . For example,

$$W_1 \in \mathbb{R}^{5d \times d}, \quad W_2 \in \mathbb{R}^{d \times 5d}.$$

2.4 Causal Masking

In an autoregressive language model, the model predicts the next word from all words before the current position:

$$w_t \sim \text{softmax}(f(w_{1:t-1})).$$

The key constraint is that the model cannot use information from future positions when predicting position t . Otherwise, it would see the answer during training.

An RNN satisfies this constraint naturally because its hidden states are computed sequentially:

$$h_{t-1} = \sigma(W h_{t-2} + U x_{t-1}), \quad w_t \sim \text{softmax}(h_{t-1} E).$$

The state h_{t-1} contains information only from time steps $1, \dots, t-1$ and cannot see the future.

Self-attention, however, allows every position to attend to the entire sequence by default. An autoregressive model therefore needs a **causal mask**, which restricts position i to positions satisfying $j \leq i$.

The mask can be added directly to the attention score:

$$s_{ij}^{\text{masked}} = \begin{cases} s_{ij}, & j \leq i, \\ -\infty, & j > i. \end{cases}$$

Applying softmax gives

$$\alpha_{ij} = \text{softmax}(s_i^{\text{masked}})_j.$$

Because $\exp(-\infty) = 0$, we have $\alpha_{ij} = 0$ whenever $j > i$. Future tokens cannot contribute to the current position.

A causal mask therefore makes Transformer self-attention satisfy the autoregressive constraint: every position can use only itself and the positions before it.

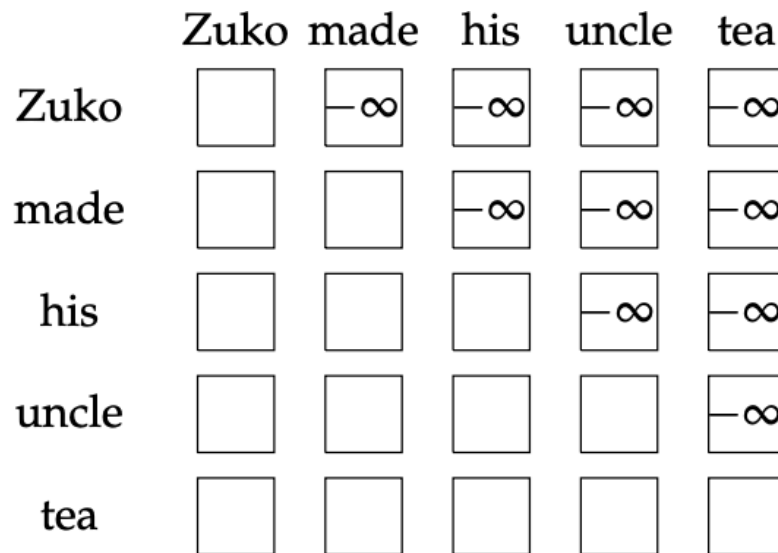


Figure 2: A causal attention mask

2.5 Summary of the Minimal Self-Attention Architecture

A basic self-attention model consists of the following components:

1. **Input representation:** Map tokens to word vectors and add positional representations so that the model can perceive token order.
2. **Self-attention:** Compute Queries, Keys, and Values from the inputs, then aggregate information from across the sequence to obtain contextualized representations.
3. **Element-wise nonlinearity:** Apply an FFN independently to every token after self-attention to increase the model's representational capacity.
4. **Causal mask:** In an autoregressive language model, restrict position i to tokens at positions $j \leq i$, preventing it from using future information.

The simplest self-attention architecture can thus be written as

word embeddings + positional representations $\rightarrow$ **self-attention** $\rightarrow$ **FFN** $\rightarrow$ **output**.

For autoregressive language modeling, a causal mask is additionally applied inside self-attention.

3 Transformer

A Transformer is an architecture based on self-attention. It consists of multiple stacked blocks, each containing a self-attention layer, a feed-forward layer, and several other components discussed below. The following diagram shows a decoder-only Transformer architecture.

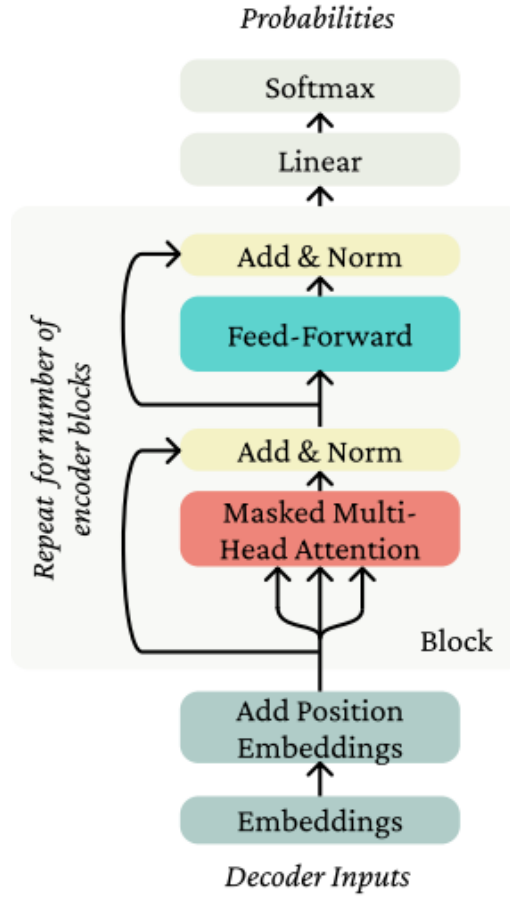


Figure 3: A decoder-only Transformer architecture

3.1 Multi-Head Self-Attention

Single-head self-attention takes one weighted average over all Values. To let a model attend to the sequence from multiple perspectives at the same time, we can run several self-attention operations in parallel. This is called **multi-head self-attention**.

Suppose there are k attention heads and let $d_h = d/k$. Each head has independent projection matrices

$$W_Q^{(\ell)}, W_K^{(\ell)}, W_V^{(\ell)} \in \mathbb{R}^{d_h \times d}.$$

Head ℓ first computes its own Queries, Keys, and Values, then performs attention independently:

$$\alpha_{ij}^{(\ell)} = \frac{\exp\left((q_i^{(\ell)})^\top k_j^{(\ell)}\right)}{\sum_{j'=1}^n \exp\left((q_i^{(\ell)})^\top k_{j'}^{(\ell)}\right)},$$

$$h_i^{(\ell)} = \sum_{j=1}^n \alpha_{ij}^{(\ell)} v_j^{(\ell)}.$$

Each head produces a d_h -dimensional output. The outputs of all heads are concatenated and passed through an output projection $W_O \in \mathbb{R}^{d \times d}$:

$$h_i = W_O \left[h_i^{(1)}; \dots; h_i^{(k)} \right].$$

Matrix form of self-attention

Let the input sequence be represented by $X \in \mathbb{R}^{n \times d}$. All Queries, Keys, and Values can be computed at once:

$$Q_X = XW_Q^T, \quad K_X = XW_K^T, \quad V_X = XW_V^T.$$

The attention scores between every pair of tokens can then be computed simultaneously:

$$A = \text{softmax}(Q_X K_X^T) \in \mathbb{R}^{n \times n}.$$

The entry A_{ij} is the attention weight from position i to position j . The final weighted sum over the Values is

$$H = AV_X = \text{softmax}(Q_X K_X^T) V_X \in \mathbb{R}^{n \times d}.$$

The following diagram illustrates these matrix operations.

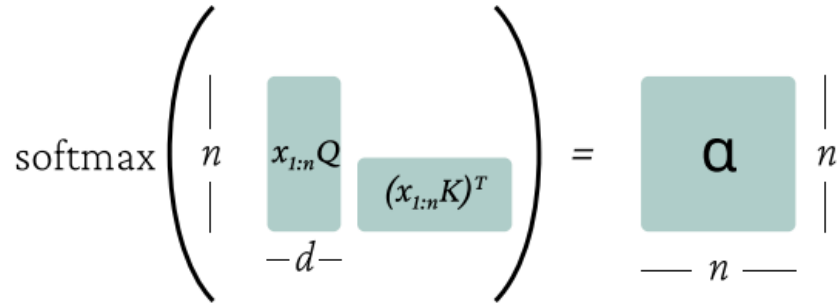


Figure 4: The matrix form of self-attention

Multi-head self-attention can also be computed in parallel through tensor reshaping. First compute $XW_Q, XW_K, XW_V \in \mathbb{R}^{n \times d}$. For k heads, split the dimension d into $k \times d_h$ and rearrange the tensors into shape $k \times n \times d_h$. The k heads can then compute their attention matrices and outputs simultaneously:

$$A = \text{softmax}(QK^T), \quad H = AV.$$

Finally, concatenate the head outputs back into a d -dimensional representation and apply one more linear transformation. Multi-head attention therefore splits one d -dimensional attention operation into k parallel d_h -dimensional operations, with a computational cost comparable to that of single-head attention at the full dimension.

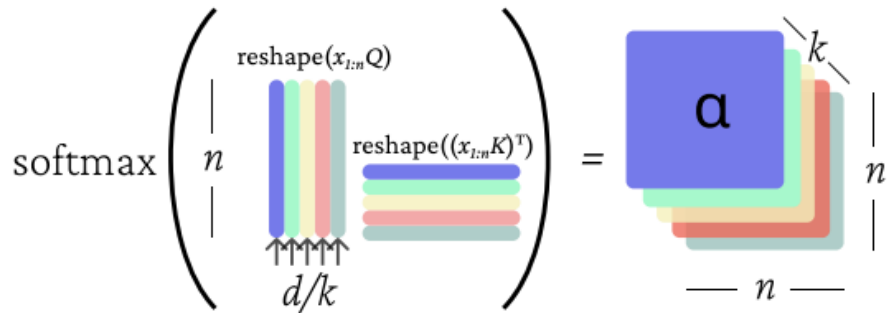


Figure 5: Parallel matrix computation for multi-head attention

3.2 Layer Normalization

Transformers commonly use layer normalization to stabilize the activations of each layer and improve gradient propagation during training.

For the hidden state $h_i \in \mathbb{R}^d$ at sequence position i , LayerNorm computes statistics only across that token's own d hidden dimensions. It does not mix information from other positions.

First compute the mean and standard deviation:

$$\hat{\mu}_i = \frac{1}{d} \sum_{j=1}^d h_{i,j}, \quad \hat{\sigma}_i = \sqrt{\frac{1}{d} \sum_{j=1}^d (h_{i,j} - \hat{\mu}_i)^2}.$$

Then normalize the representation:

$$\text{LN}(h_i) = \frac{h_i - \hat{\mu}_i}{\hat{\sigma}_i}.$$

In practice, an implementation also includes a small numerical-stability term as well as learnable scale and bias parameters, allowing the model to readjust the normalized representation.

The main difference from BatchNorm is that LayerNorm does not depend on other examples in the batch. This makes it particularly suitable for Transformers and other sequence models.

3.3 Residual Connections

A residual connection adds the input of a layer directly to its output:

$$h' = f(h) + h.$$

This provides a more direct path for gradients and makes deep networks easier to train.

In a Transformer, residual connections are usually combined with LayerNorm in an Add & Norm operation. Two common arrangements are:

- **Pre-Norm:** Normalize first, pass the result through the sublayer, and then add the residual connection.

$$h_{\text{pre-norm}} = f(\text{LN}(h)) + h.$$

- **Post-Norm:** Pass the input through the sublayer, add the residual connection, and then normalize.

$$h_{\text{post-norm}} = \text{LN}(f(h) + h).$$

Here, f may be either self-attention or an FFN. In practice, Pre-Norm usually provides more stable gradient propagation.

3.4 Attention Logit Scaling

Attention uses the dot product of a Query and Key to measure their relevance. As the dimension d increases, the magnitude of the dot product tends to grow, which can make the softmax distribution excessively sharp and hinder gradient propagation.

Scaled dot-product attention divides the score by $\sqrt{d}$:

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right).$$

The result then weights the Values:

$$H = AV = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V.$$

In multi-head attention, each head usually has dimension $d_h = d/k$, so the actual scaling factor is $\sqrt{d_h}$.

3.5 Transformer Encoder

A Transformer Encoder receives the complete input sequence and therefore does not require a causal mask.

First, convert the tokens to embeddings and add positional representations:

$$H^{(0)} = X + P.$$

Then stack several Encoder Blocks. Each Encoder Block contains two main sublayers:

1. **Multi-head self-attention:** Allows information to move between different positions.
2. **FFN:** Applies an independent nonlinear transformation at every position.

Both sublayers are combined with residual connections and LayerNorm. Using Pre-Norm as an example,

$$H' = H + \text{MHA}(\text{LN}(H)),$$

$$H_{\text{out}} = H' + \text{FFN}(\text{LN}(H')).$$

After several Encoder Blocks are stacked, the model produces a contextualized representation for every token.

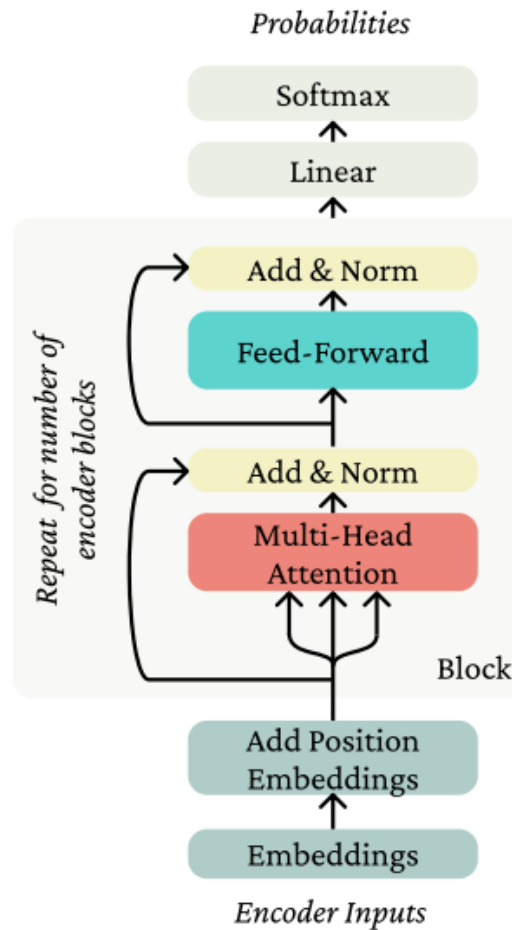


Figure 6: A Transformer Encoder architecture

A Transformer Encoder has no causal mask, so every position can attend to the entire input sequence. It is well suited to tasks that require a contextualized representation of the complete sequence.

Its main properties are:

- Every token can attend to every position in the sequence.
- Each output representation incorporates both the current token and its surrounding context.
- It is more naturally suited to sequence understanding than to autoregressive text generation.

3.6 Transformer Decoder

A Transformer Decoder is primarily used for autoregressive language modeling. Its main difference from an Encoder is that Decoder self-attention uses a causal mask, so position i can see only itself and earlier positions.

When generating the t -th word, the model can therefore condition only on $w_{1:t-1}$:

$$w_t \sim \text{softmax}(f(w_{1:t-1})).$$

Stacking multiple Decoder Blocks produces an autoregressive Transformer.

3.7 Transformer Encoder-Decoder

An Encoder-Decoder Transformer operates on two sequences. The input sequence first passes through the Encoder to obtain contextualized representations:

$$H^{(x)} = \text{TransformerEncoder}(x_{1:n}).$$

The Decoder applies causally masked self-attention to the target sequence and uses **cross-attention** to read the Encoder output.

Cross-attention has the same computational form as self-attention, but its Queries, Keys, and Values come from different sequences:

$$q_i = W_Q h_i^{(y)}, \quad k_j = W_K h_j^{(x)}, \quad v_j = W_V h_j^{(x)}.$$

The Queries come from the Decoder, while the Keys and Values come from the Encoder. The attention weights are

$$\alpha_{ij} = \frac{\exp(q_i^\top k_j)}{\sum_{j'=1}^n \exp(q_i^\top k_{j'})}.$$

The Decoder then takes a weighted sum over the Encoder Values:

$$h_i = \sum_{j=1}^n \alpha_{ij} v_j.$$

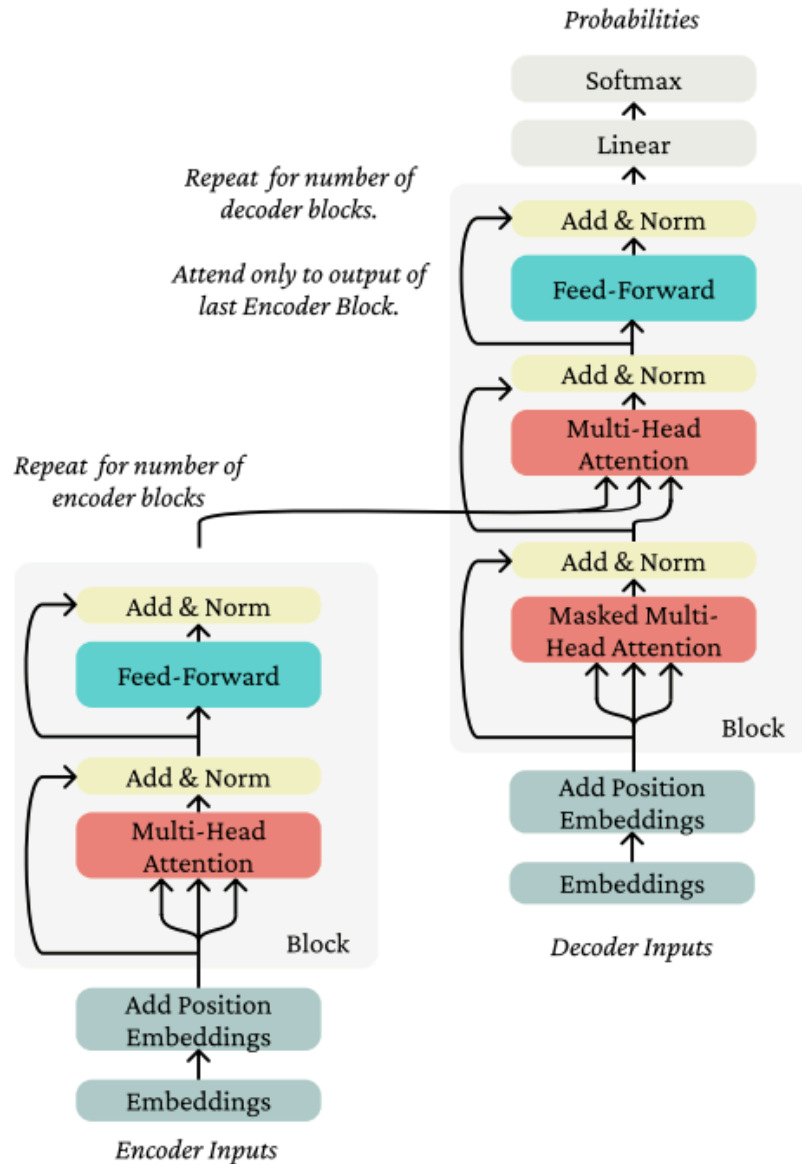


Figure 7: A Transformer Encoder-Decoder architecture

An Encoder-Decoder architecture is useful when a task requires both bidirectional understanding of an input and autoregressive generation of an output.

- The Encoder allows every input token to attend to the entire input sequence, producing complete contextualized representations.
- The Decoder predicts the output one step at a time from the content already generated.
- Cross-attention lets the Decoder read the Encoder representations during generation.

Encoder-Decoder Transformers are well suited to sequence-to-sequence tasks such as summarization and translation.

However, this architecture must divide model parameters between the Encoder and Decoder. As models grow larger, a Decoder-only architecture is often simpler, which is why many large language models use Decoder-only Transformers.